A Skill-Level Progress Tracker for Stellar Education

Godsgift Braimah, Maïlys Guedon, Seungmyun Park, Siting Wang

2026-06-23

Table of contents

1 Executive summary

A homework score is a poor diagnostic tool. In fact, report-card grades tell a parent or teacher *how much* a student got right but say nothing about *which* skills are behind. This discrepancy is what Stellar Education strives to remedy; to give teachers, students, and parents a clear, skill-by-skill view of where each student stands.

We implemented this by first converting raw AP Statistics homework responses into mastery probabilities for each student, per-skill using **Bayesian Knowledge Tracing (BKT)**, an established method for inferring whether a learner has genuinely grasped a skill from the pattern of their responses over time (Corbett and Anderson 1994). Then, we rendered those estimates through an interactive **Shiny** dashboard built around two audiences: a teacher view for catching at-risk students early, and a student view that turns the same numbers into a personalised plan, highlighting *what to work on next*.

2 Introduction

Stellar Education is a startup that runs small-group private tutoring for students and aims to replace generic instructions with something genuinely tailored to each learner. Achieving this requires knowing, with real precision, what a student does or does not understand.

A 70% quiz score only shows that three out of ten questions were missed, it doesn't reveal which skills were lacking, whether those skills are critical for future material, or how a tutor should adjust instruction. Bridging the gap between a grade and actionable insight became the central goal of this project.

We designed the product around three objectives, informed by the needs of our partner:

1. **Reliable insight.** Estimate mastery for every student–skill pair using a probabilistic model that accounts for factors such as guessing and careless mistakes.
2. **Instant visibility.** Present mastery estimates through an interface that teachers can interpret at a glance, and giving students targeted guidance beyond simply reviewing material.
3. **Timely intervention.** Prioritize the skills most in need of attention so meaningful learning gaps can be addressed early.

To achieve this, we use the **Knowledge Component (KC)** as our primary unit of analysis. **A KC is a specific skill** required to complete a task and is more granular than a topic and more precise than a subject, isolating exactly what a student must know.

For example, a question involving **measures of central tendency** may appear as a single task, but it depends on **three separate KCs: finding the mean, median, and mode.** A student may master one of these skills while struggling with the others. Analyzing performance at the KC level moves beyond a single aggregate score. Instead of reporting that a student earned 70%, the model can identify which skills have been mastered, which are developing, and which remain uncertain, providing the information needed to determine the most effective next step.

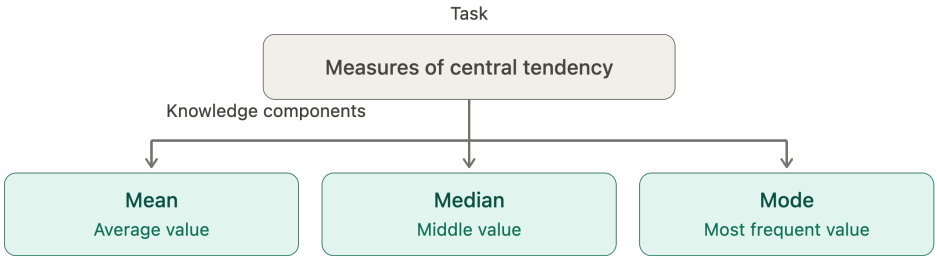

Figure 1: Measures of central tendency decomposed into its underlying knowledge components

3 Data

Stellar Education provided homework response data from 25 AP Statistics students across 10 instructional units and 47 modeled knowledge components. Responses were recorded between early September 2025 and April 2026. They also supplied KC relationship data (KC Nodes and KC Edges) describing prerequisite, dependency, and supporting relationships between knowledge components.

Each row in the raw dataset represents a student's response to a single assessment item. Every item is mapped to the knowledge component (KC) it assesses, allowing performance to be analyzed at either the item, skill, or student level.

Because multiple items may assess the same KC, we grouped responses by student-KC pair for modeling. Using `order_id` to maintain the sequence of attempts, we created a time series of responses for each student on each KC. This ordered sequence is the input to the mastery model, which estimates the probability that a student has mastered a given KC over time.

3.1 Fields used by the model and dashboard

The final dataset output from our pipeline, `final_student_kc_data.csv`, contains many variables, but only the following are used directly by the model and dashboard:

- `student_id`: Anonymized student identifier (S001–S025).
- `modeling_kc_id` / `modeling_kc_label`: The knowledge component being assessed and its human-readable label.
- `order_id` / `kc_attempt`: Sequence indicators that preserve the order of observations for each student-KC pair.
- `correct`: Binary outcome indicating whether the response was correct.
- `state_predictions`: The model's estimated mastery probability, which drives the dashboard's mastery indicators and progress visualizations.
- `unit`: Groups knowledge components into the ten AP Statistics units used in the Unit Mastery view.

3.2 Knowledge Component Structure

The dataset represents knowledge at two levels of granularity. Each homework responsecis tagged with a **fine-grained KC (`fine_kc_id` / `fine_kc_label`)**, which captures the specific concept being tested, and a **modeling KC (`modeling_kc_id` / `modeling_kc_label`)**, which groups related concepts into a broader skill. We estimated our mastery probabilities at the modeling-KC level, allowing evidence from multiple related items to be combined and reducing sparsity in the data.

3.3 Curriculum Metadata and Data Considerations

In addition to student responses, Stellar Education provided metadata describing the relative importance of each KC. Our team used this information to prioritize skills in the dashboard, ensuring that foundational concepts and exam-relevant topics receive greater attention than isolated or low-impact skills.

3.3.1 Data Quality and Ethics

We included data considerations and feedback from our Capstone Partner into the design process.

- **Sparse coverage.** We discovered that not every student attempted every KC, so the student-by-KC matrix is incomplete. The dashboard treats absent attempts as "not started" rather than as failures.
- **Privacy.** Students are identified only by anonymized codes (S001–S025); no personally identifying information is used or displayed.
- **Mixed-type columns.** A revision-note column (`2026_27_revision_note`) loaded with mixed types and was excluded from modeling; it carries no signal for mastery estimation.

4 Data Science Methods

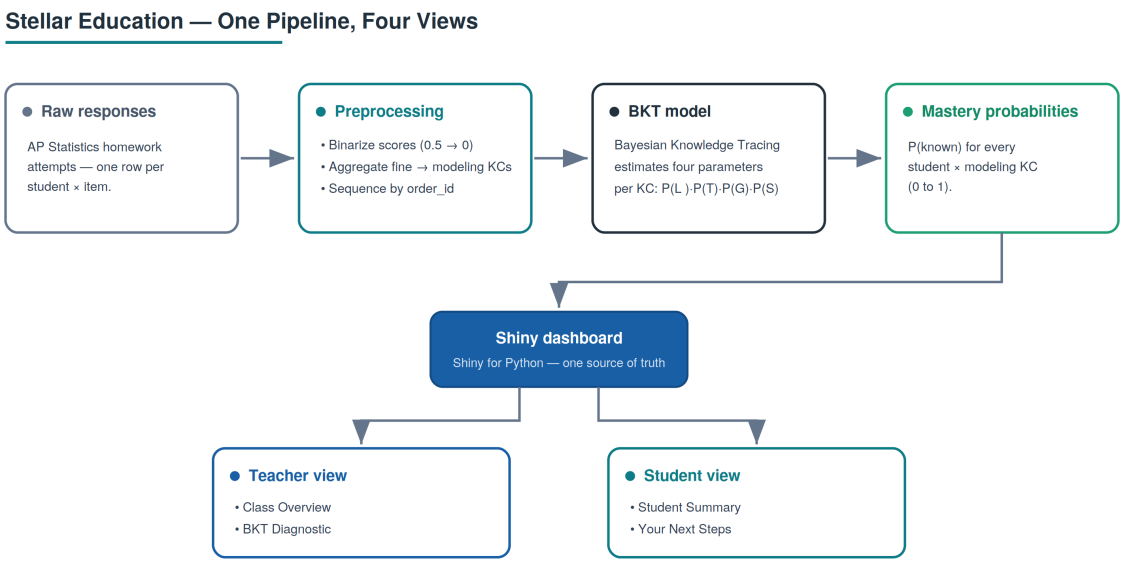

Figure 2: From raw responses to a teacher and student-facing dashboard.

4.1 Why Bayesian Knowledge Tracing (BKT)?

We began our model selection with a simple question: **What does Stellar Education actually need this system to do?**

The goal was not to grade a class that has already ended, it is to guide data collection for future cohorts, deciding which knowledge components to assess next and which students need more practice on which skills. This is a forward-looking problem, and it called for a method that revises its estimate of a student's mastery as new attempts to questions arrive over the course of the term, so that our recommendations always reflect where each student stands now.

BKT met that need, and we chose it over the other alternatives for four major reasons:

1. **It works on small cohorts.** Deep-learning approaches to knowledge tracing such as Deep Knowledge Tracing (DKT) typically need thousands of observations to train reliably and would overfit on our data. BKT estimates only four parameters per knowledge component, so it produces meaningful estimates from a class of 25 students across 47 KCs (Badrinath, Wang, and Pardos 2021).
2. **It matches the structure of our data.** We have student responses to questions, on a skill, recorded in the order it happened, this is precisely the sequential input BKT was designed to model.
3. **It updates without retraining.** Each new response refines a student's mastery estimate immediately, which lets us track students continuously across the term and use the most recent estimate to decide what to assess next.
4. **Its output is interpretable.** Mastery is expressed as a single probability between 0 and 1, so Stellar Education can read it directly; "this student has a 73% chance of having mastered this skill," and act on it.

4.2 BKT Methodology

The core modeling question is: *given everything a student has answered so far, what is the probability they have actually mastered this skill?* BKT answers this by treating mastery as a hidden state that is updated after each new observation. It continuously refines this belief using **four interpretable parameters**, estimated separately for each knowledge component (KC) (Corbett and Anderson 1994; Bulut, Shin, and Cormier 2023).

1. **P(L0): Prior knowledge.** The probability a student already possessed the skill before any practice took place.
2. **P(T): Learning rate.** The probability that, following an attempt, a student transitions from *not knowing* the skill to *knowing* it.
3. **P(G): Guess.** The probability of answering correctly without having mastered the skill.
4. **P(S): Slip.** The probability of answering incorrectly despite having mastered the skill.

The four parameters describe the skill, not the student. To track a student, BKT keeps a running score called P(Known), the chance the student has learned the skill. It starts at the initial-knowledge value [P(L)] and updates after every question in two steps.

First, it adjusts the score based on whether the student got the question right or wrong, using the chance of a lucky guess [P(G)] and the chance of a careless slip [P(S)]. Then it nudges the score up a little to account for the chance the student just learned something from that attempt [P(T)]. The new score carries over to the next question, and so on. **That running score is our mastery probability.**

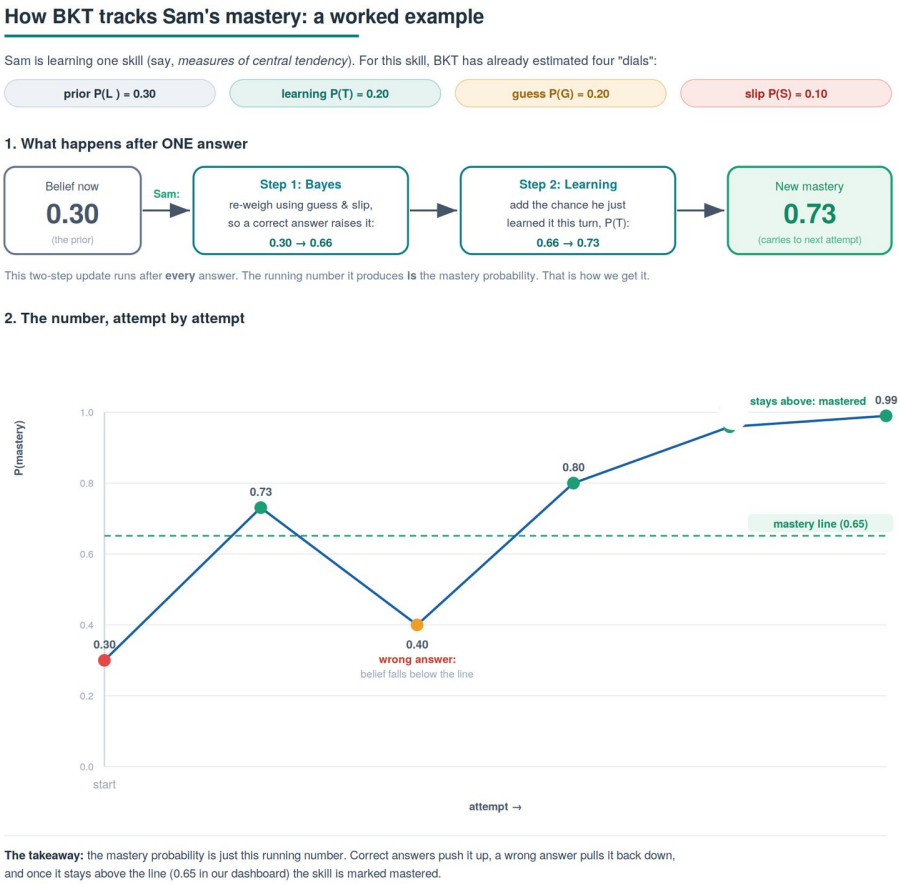

Figure 3: For a student Sam, learning the Measures of Central Tendency, BKT infers a hidden mastery state from observed answers, mediated by guess and slip.

4.3 Feature Engineering and Preprocessing

4.3.1 Data Simulation

In early experiments, we found that the dataset was too sparse for reliable BKT estimation, with many students having only one to three attempts per knowledge component. Under these conditions, the model produced unstable or degenerate parameter estimates, as there was insufficient information to distinguish learning from guessing and slipping.

To test whether the issue came from the model or the data, we created a synthetic dataset that preserved the structure of the course but increased the number of attempts per KC. The simulation used the same KC set and assigned plausible BKT parameters, then generated 200 student learning sequences with 8–15 attempts per KC. This confirmed that the earlier failures were due to sparsity in the real data rather than issues with the model itself.

4.3.2 BKT Implementation Comparison

To verify that BKT's weak performance was a property of the model itself and not an artifact of a particular implementation, we fit the same dataset using three independent BKT implementations: `pyBKT` using google colab, a custom-built `manual_bkt`, and `LearnSphere's BKT pipeline`. `manual_bkt` was developed after `pyBKT` (running locally) consistently returned degenerate parameters (prior = NaN, learns = 1.0, guess = slip = 0.5) on both the real dataset and synthetic data with known ground truth.

All three produced similar results, with parameters converging to comparable values (guess 0.3, slip 0.2–0.3) and item-level performance (AUC 0.50–0.57). Since LearnSphere runs on a separate codebase and reproduced the same behavior, we concluded that the limitation came from the data rather than the software.

4.3.3 KC Aggregation and Structural Constraints

We found that the data violated two key requirements for stable BKT estimation: sufficient `vertical sequence` (multiple attempts per student–KC pair) and `horizontal depth` (enough students per KC). Many KCs had only a few student interactions, making reliable estimation difficult.

To address this, we reduced granularity by aggregating fine-grained KCs into higher-level modeling KCs using a curriculum map provided by our partner. For example, multiple KCs within descriptive statistics were grouped into a single modeling KC. This significantly increased the number of observations per KC and reduced the total number of modeled skills.

4.3.4 Data Aggregation and Final Design Choice

We also explored alternative grouping strategies, including grouping by prerequisite structure, graph proximity, and domain-expert clustering. All three of these approaches helped to reduce the number of groups to predict (over 80% reduction in the number of groups compared to the initial 256) as well as increase the number of observations per group.

When comparing the performance of the three approaches to a baseline (initial fine KCs), we found that the BKT predictions were marginally better. However, having fewer groups helped us when designing and choosing visualization to include in the dashboard.

Finally, since BKT updates mastery after every attempt, we retain only the most recent estimate per student–KC pair as the current mastery value used in the dashboard.

4.4 Evaluation

To assess whether the model generalises to unseen students, we used a student-level train-test split, `70%` of students for training, `30%` held out for testing. Splitting by student rather than by attempt reflects real deployment conditions and prevents data leakage.

We report two complementary metrics. `RMSE` measures the average gap between the model's predicted probability of a correct response and the actual outcome, so it captures how well-calibrated the predictions are (lower is better). `AUC` measures whether the model correctly ranks struggling students below performing ones (1.0 is perfect, 0.5 is chance).

Together, **RMSE asks whether the predicted probabilities are numerically accurate, while AUC asks whether the model ranks students correctly, placing those who are actually struggling below those who are doing well.** because a model can do well on one and poorly on the other.

4.5 Ethical Considerations and Limitations of our Approach

1. BKT models mastery as a **two-state** variable, a skill is either mastered or not, but real understanding falls somewhere in between, so partial knowledge is missed.

2. BKT also treats mastery as **permanent**, so it does not model forgetting between sessions (Bulut, Shin, and Cormier 2023). Because these estimates affect which students a tutor focuses on, they should **support that decision rather than replace the tutor's own judgment**, especially for students with very few recorded attempts, whose scores are the least reliable.

5 Data Product and Results

Our deliverable includes the mastery probabilities rendered as an interactive dashboard, built with Shiny for Python (Posit Software, PBC 2024), that turns the per-student, per-KC mastery probabilities into a view a teacher can act on. We organized it into two views that answer two different questions. `Class Overview` asks how the whole class is doing, and `Student Overview` asks how one student is doing.

5.1 Class Overview

The Class Overview opens with four summary boxes: the total number of KCs and how many are currently Mastered, Progressing, or Needing Practice, each compared against the cohort's first attempt to show movement over time. Below them, three panels give complementary views of the same cohort.

- **Unit Mastery Overview**. A grid where every KC is a coloured tile grouped by unit, green for mastered, yellow for progressing, and red for needs practice. It makes weak units visible at a glance.
- **KC Progress**. For a selected KC, we show the class student by student alongside the percentage who have mastered that skill. Tabs separate the most important KCs, those needing practice, and those showing good progress.
- **KC Opportunities and Opportunity Heatmap**. These show how much practice each skill has received. The heatmap plots students against KCs and shades each cell by practice volume, exposing skills the whole class has barely attempted.

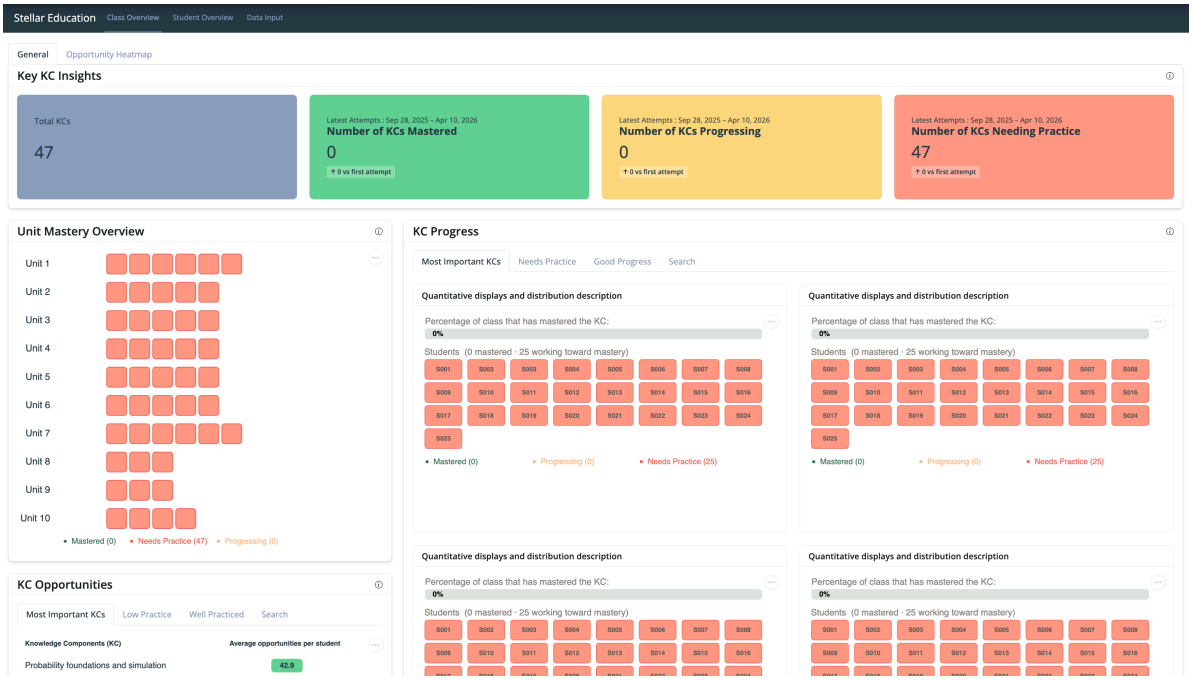

Figure 4: Teacher View 1 with Class Overview: Whole-class status across all knowledge components.

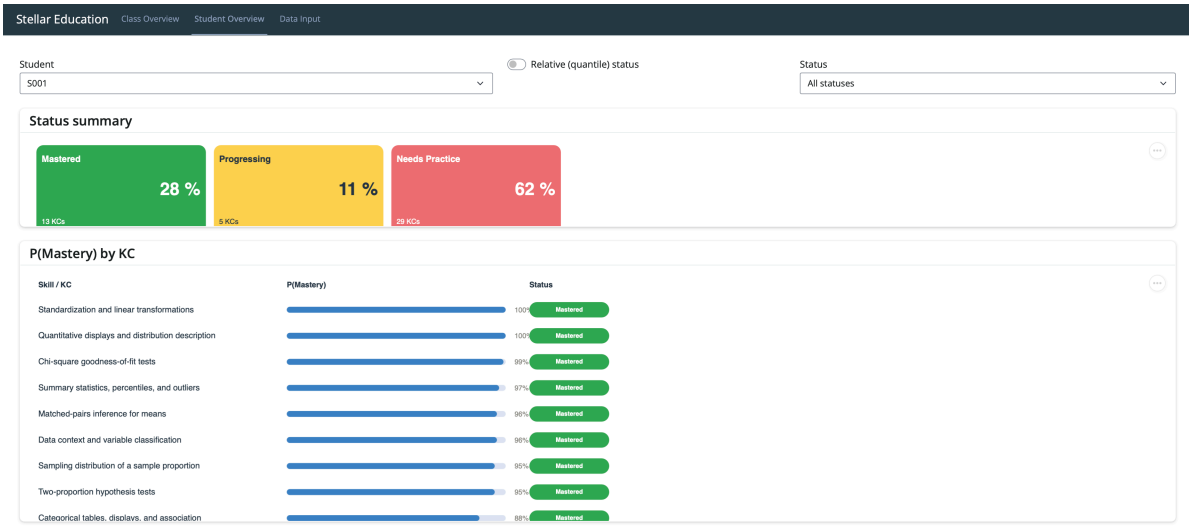

Figure 5: Teacher View 2 with Student Specifics: Drilling into a Single Student in the class.

5.2 Student Overview

The Student Overview focuses on one learner. A status summary shows the share of that student's KCs that are Mastered, Progressing, or Needing Practice, and a detailed table lists every KC with its mastery probability as a progress bar and a coloured status badge. Two controls shape the view: a status filter, and a toggle between absolute and relative grading.

A student can access their next steps based on their current mastery levels. The prerequisite map shows the student's current mastery for each KC, with arrows indicating which skills are required for others.

Status thresholds. We assign each KC a status from its mastery probability using fixed cut points:

Status	Mastery probability
Mastered	≥ 0.65
Progressing	$0.35 - 0.65$
Needs Practice	< 0.35

Absolute vs. Relative status. In absolute mode the thresholds above are applied directly, answering "is this skill objectively mastered?" In relative (quantile) mode the same KCs are graded against the whole class's distribution, answering "where does this student rank among peers?" A strong student therefore stays mostly green under the relative view, while a struggling student's KCs are spread across the three bands rather than all flagged red.

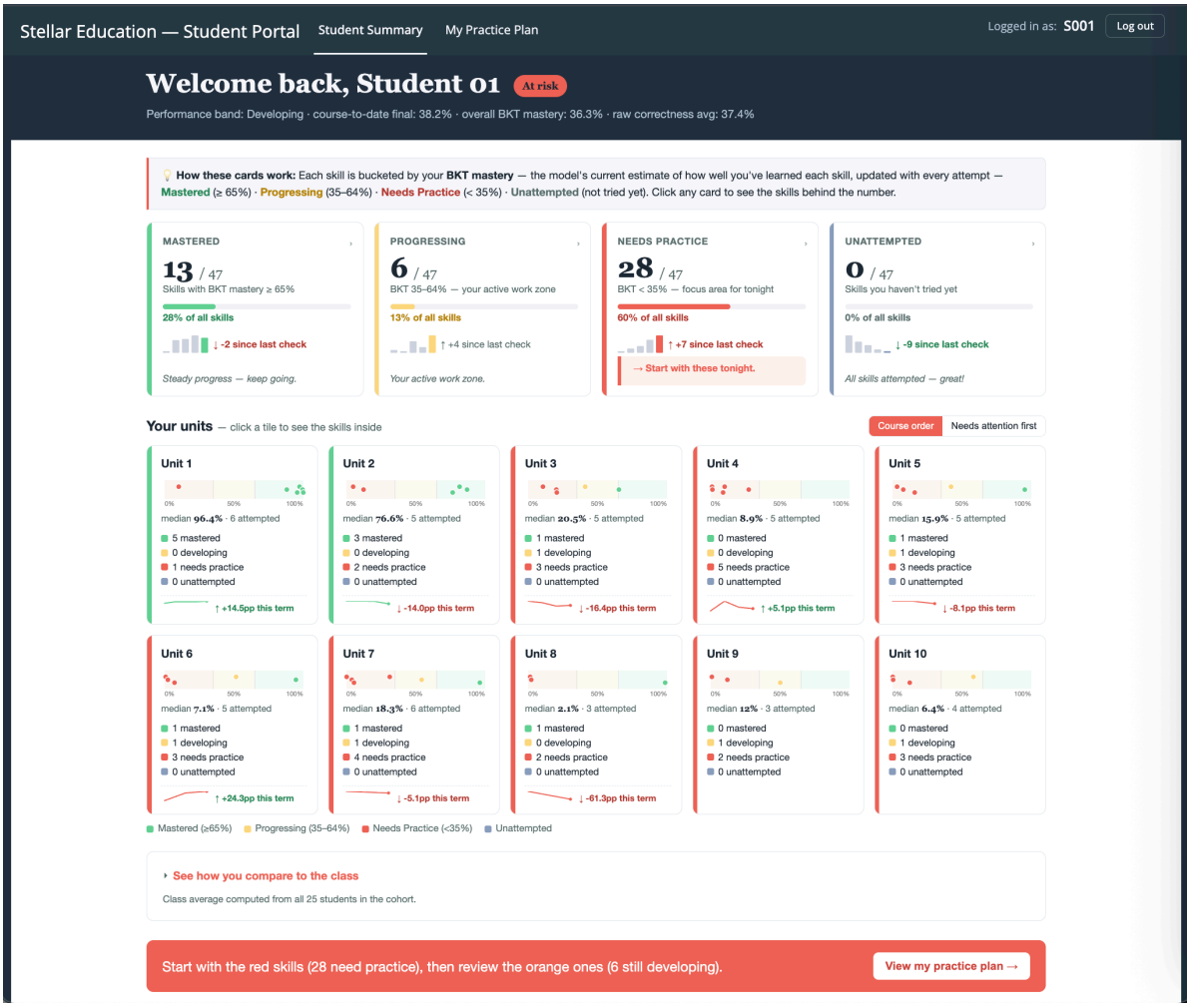

Figure 6: Student Summary: a learner's own progress, skill by skill.

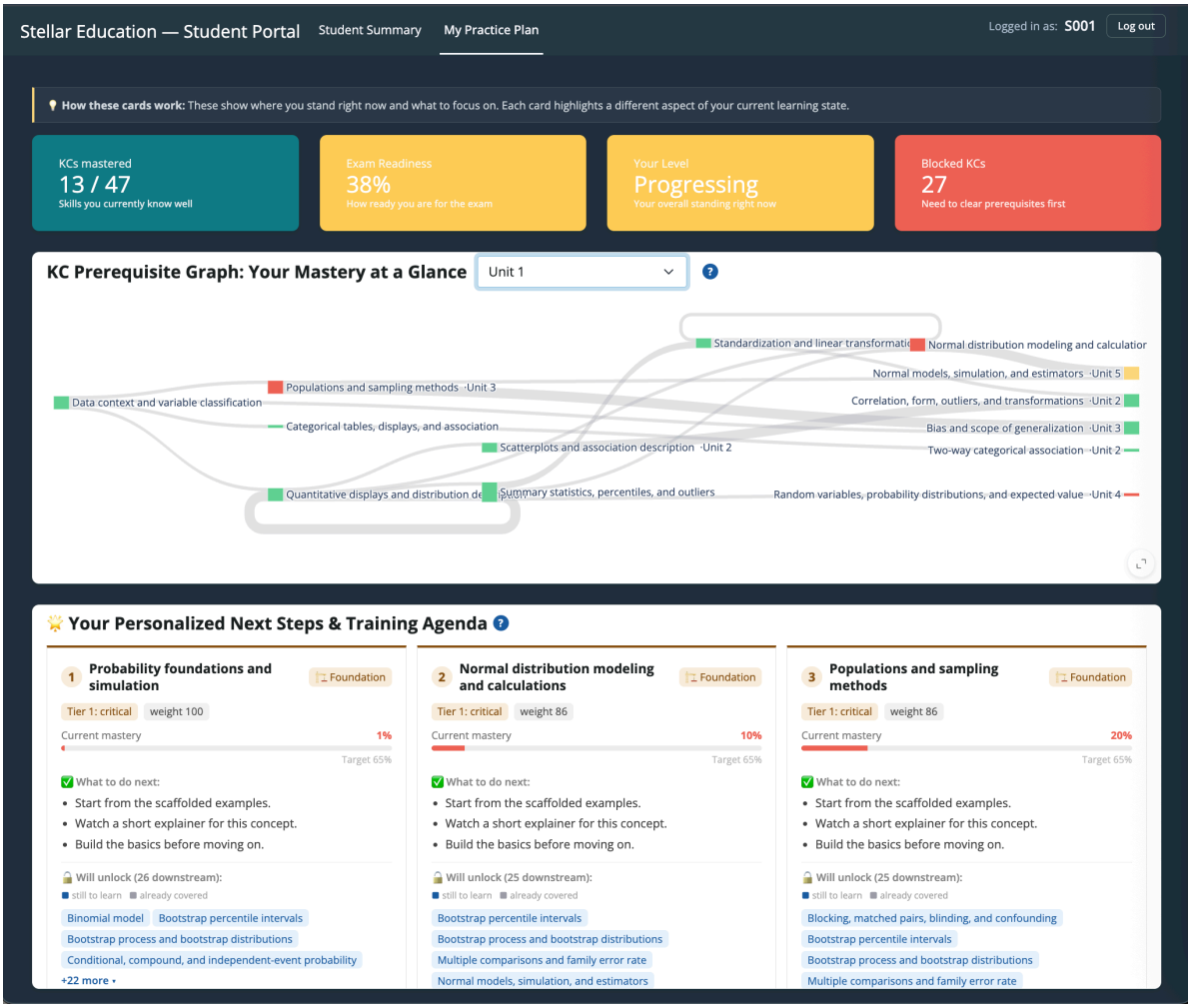

Figure 7: Your Next Steps: weight-aware recommendations with a prerequisite map.

5.3 Results

We ultimately delivered two main outcomes. The first is a functional dashboard that translates raw data into stable and easy to read mastery estimates for each Knowledge Component (KC). Teachers can now look at a single screen to identify weak units, spot under practiced skills, and see exactly how each student is progressing.

The second outcome is a specific data collection target, which is just as important for Stellar moving forward. Our modeling revealed that mastery estimates fluctuate wildly with only two or three attempts per skill, but they stabilize around eight to fifteen attempts. This is not just a casual observation; it is a baseline specification that Stellar can use to design future assessments.

6 Conclusions and Recommendations

A dashboard is only as good as the data feeding it. To ensure the mastery views remain reliable, we recommend the following:

1. **Aim for at least eight attempts per KC:** Future assessments should ensure each student practices a modeled knowledge component eight to fifteen times. Any mastery estimates based on fewer than eight attempts should be treated as provisional estimates rather than hard facts.
2. **Focus on critical skills first:** Instead of spreading practice evenly, use the existing tier and exam weighting metadata to ensure critical KCs are prioritized.
3. **Follow the prerequisite structure:** Since foundational skills naturally impact downstream learning, gathering data on those base KCs first will give you the most diagnostic value.
4. **Run a validation check:** Once you have collected a richer dataset, test these mastery estimates against an independent metric like a final exam to confirm they actually predict real performance.

6.1 Limitations

• **Targets based on simulation:** While the eight to fifteen attempt rule relies on actual KC structures, it was derived through simulation. It needs to be tested and confirmed once real high-volume cohort data comes in.

• **Small sample size:** Our current estimates are based on just 25 students in a single course. They should not be blindly applied to other subjects or cohorts without adjusting the model.

• **Unconfirmed predictive validity:** As mentioned above, we still need to validate these mastery estimates against independent outcomes before we can fully vouch for their predictive power.

7 References

Badrinath, Anirudhan, Frederic Wang, and Zachary Pardos. 2021. "pyBKT: An Accessible Python Library of Bayesian Knowledge Tracing Models." In *Proceedings of the 14th International Conference on Educational Data Mining (EDM)*. https://doi.org/10.48550/arXiv.2105.00385.

Bulut, Okan, Jinnie Shin, and Damien C. Cormier. 2023. "An Introduction to Bayesian Knowledge Tracing with pyBKT." *Psych* 5 (3): 770–86. https://doi.org/10.3390/psych5030050.

Corbett, Albert T., and John R. Anderson. 1994. "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge." *User Modeling and User-Adapted Interaction* 4 (4): 253–78. https://doi.org/10.1007/BF01099821.

Posit Software, PBC. 2024. "Shiny for Python." https://shiny.posit.co/py/.

8 Appendix

Full code, the reproducible pipeline, and product documentation are available in the project repository: GitHub link.